

NumPy Fundamentals

Data Science Mentorship Program | Verified Code & Outputs

1. Introduction to NumPy

NumPy (Numerical Python) is a fundamental package for scientific computing in Python. It provides high-performance multidimensional array objects (`ndarray`). It is written in C but wrapped in Python, providing the best of both worlds: C's speed and Python's easy syntax.

- **Fixed Size:** Unlike Python lists, NumPy arrays have a fixed size upon creation.
- **Homogeneous Data:** All elements must be of the same data type.
- **Memory & Speed:** Uses contiguous memory, avoiding reference overhead of standard Python lists.

2. Creating Arrays

Various methods exist to initialize NumPy arrays.

```
import numpy as np

# 1D Array (Vector)
print(np.array([1, 2, 3]))
```

OUTPUT

```
[1 2 3]
```

```
# np.arange(start, stop, step) - Similar to Python range
print(np.arange(1, 10, 2))
```

OUTPUT

```
[1 3 5 7 9]
```

```
# Arrays with predefined values
print(np.ones((2, 2))) # 2x2 matrix of 1s
print(np.zeros((2, 2))) # 2x2 matrix of 0s
```

OUTPUT

```
[[1. 1.]  
 [1. 1.]]  
[[0. 0.]  
 [0. 0.]]
```

```
# np.linspace(start, stop, num_items) - Linearly spaced elements  
print(np.linspace(-10, 10, 5))
```

OUTPUT

```
[-10.  -5.   0.   5.  10.]
```

3. Array Attributes

Attributes describe the metadata of an array.

```
A = np.arange(1, 13).reshape(3, 4)  
  
print(A.ndim)    # Number of dimensions  
print(A.shape)   # Dimension sizes  
print(A.size)    # Total number of elements  
print(A.itemsize)# Memory occupied by a single element (in bytes)  
print(A.dtype)   # Data type of elements
```

OUTPUT

```
2  
(3, 4)  
12  
8  
int64
```

4. Changing Data Types

Use `astype()` to convert arrays to different data types, often to optimize memory.

```
A_float = A.astype(np.float32)  
print(A_float.dtype)
```

OUTPUT

float32

5. Array Operations

NumPy allows scalar (array + single number) and vector (array + array) operations seamlessly.

```
A = np.array([1, 2, 3])  
  
# Scalar Operation (element-wise)  
print(A * 2)  
print(A > 1)
```

OUTPUT

```
[2 4 6]  
[False True True]
```

6. Mathematical Functions

```
A = np.array([[1, 2], [3, 4]])  
  
# Aggregate Functions along axes (axis=0 is column, axis=1 is row)  
print(np.max(A, axis=1))
```

OUTPUT

```
[2 4]
```

```
# Dot Product (Inner dimensions must match)  
B = np.array([[1, 2], [3, 4]])  
print(np.dot(A, B))
```

OUTPUT

```
[[ 7 10]  
 [15 22]]
```

7. Indexing & Slicing

Access items in n-dimensional arrays using row and column indices: `array[row_slice, col_slice]`

```
A = np.arange(1, 17).reshape(4, 4)
# [[ 1  2  3  4]
#   [ 5  6  7  8]
#   [ 9 10 11 12]
#  [13 14 15 16]]

# Slicing the first row, all columns
print(A[0, :])
```

OUTPUT

```
[1 2 3 4]
```

```
# Custom Slice: Rows 1 to end, Columns 1 to 2
print(A[1:, 1:3])
```

OUTPUT

```
[[ 6  7]
 [10 11]
 [14 15]]
```

8. Reshaping Functions

```
# np.transpose() or .T - Swaps rows and columns
B = np.array([[1, 2], [3, 4]])
print(B.T)
```

OUTPUT

```
[[1 3]
 [2 4]]
```

```
# np.ravel() - Flattens any n-D array into a 1D array
print(np.ravel(B))
```

OUTPUT

```
[1 2 3 4]
```

9. Stacking and Splitting

Combining and separating arrays dynamically.

```
X = np.array([1, 2])
Y = np.array([3, 4])

# hstack: Horizontal stack (side-by-side)
print(np.hstack((X, Y)))

# vstack: Vertical stack (top-to-bottom)
print(np.vstack((X, Y)))
```

OUTPUT

```
[1 2 3 4]
```

```
[[1 2]
 [3 4]]
```